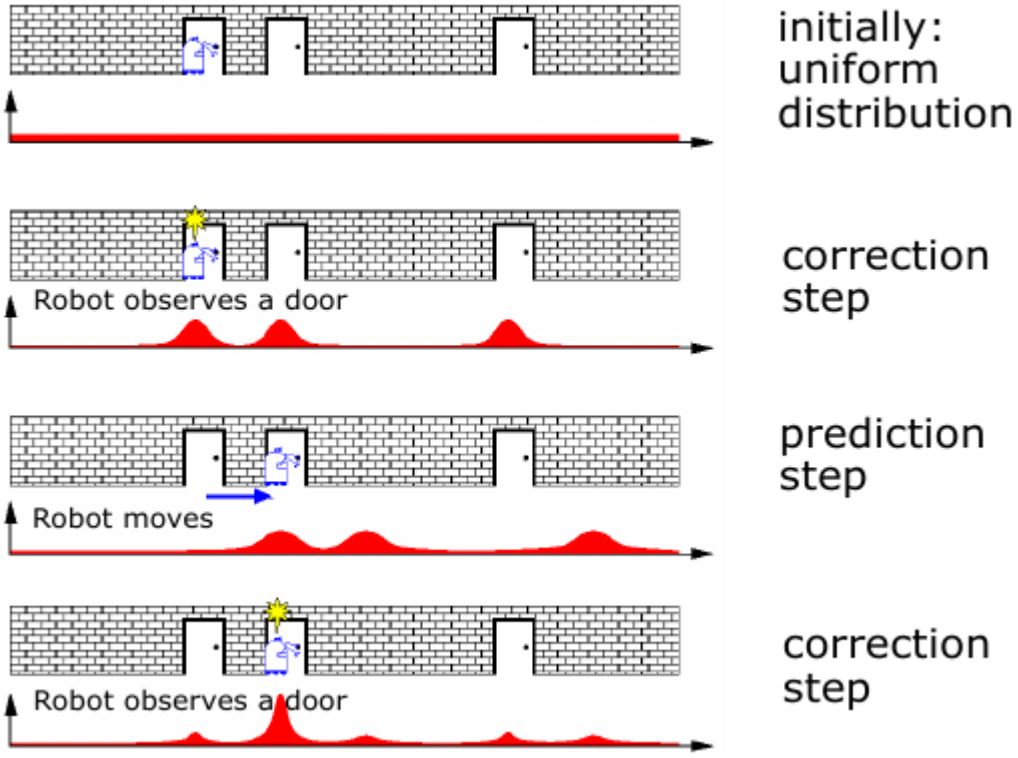
Ro11: Non-Parametric Filter for Localisation

- 2 implementations of the Bayes Filter
- **Non-parametric** filters
  - Realise probabilitstic localisation with **multimodal** distributions
  - Enable **global** localisation
  - Process **raw** range measurements
  - Use the motion and observation models presented in the previous chapter

Probabilitstic Localisation

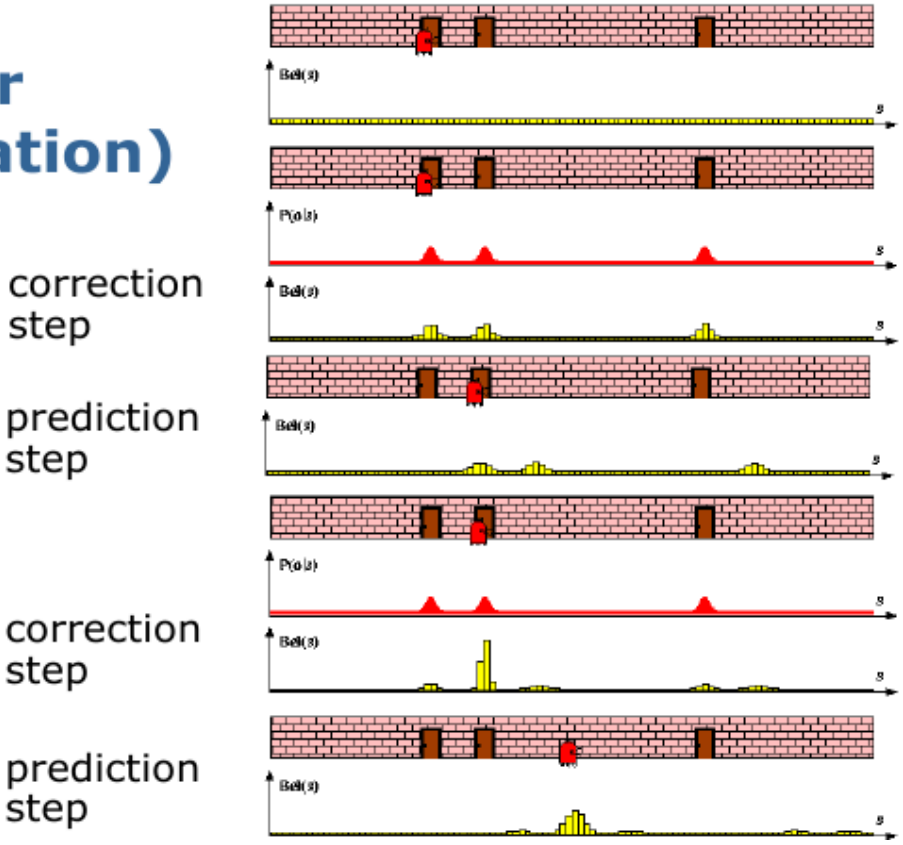


Multimodel Belief Distribution

- **Represent** & **efficiently update** multimodel posterior?
  - **Discrete filter** (Grid/Markov Localisation)
  - **Particle filter** (Monte Carlo Localisation)

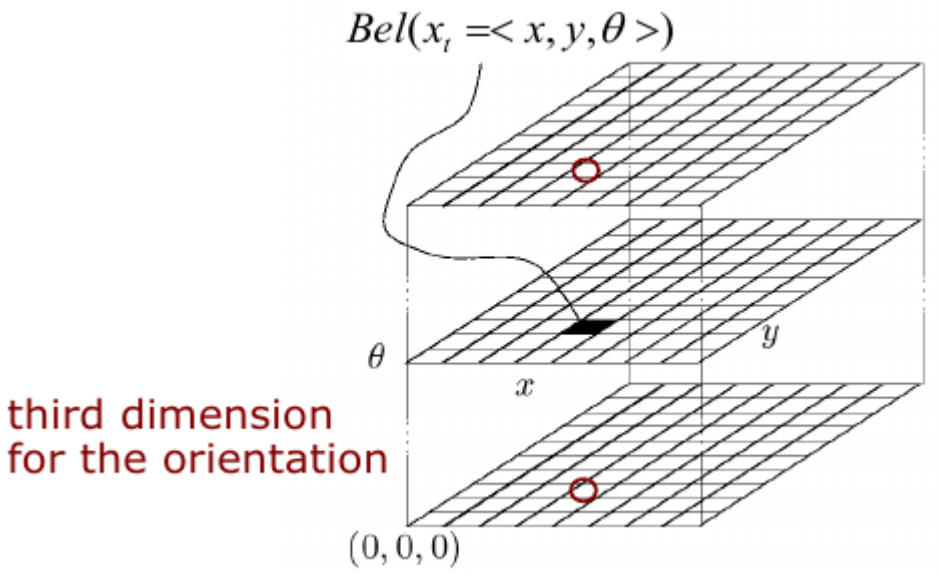
Discrete Bayes Filter

Discrete Filter (Grid Localization)



1. Algorithm **Discrete_Bayes_filter**(*Bel(x),d*):
2. η=0
3. If *d* is a **perceptual** data item *z* then
4. For all *x* do
5. $$Bel'(x) = P(z \mid x) Bel(x)$$ observation model
6. $$\eta = \eta + Bel'(x)$$
7. For all *x* do
8. $$Bel'(x) = \eta^{-1} Bel'(x)$$ normalization
9. Else if *d* is an **action** data item *u* then
10. For all *x* do
11. $$Bel'(x) = \sum_{x'} P(x \mid u, x') \, Bel(x')$$ sum over all **discrete** states; motion model
12. Return *Bel'(x)*

Piecewise Constant Representation



Processing Complexity reducing Techniques

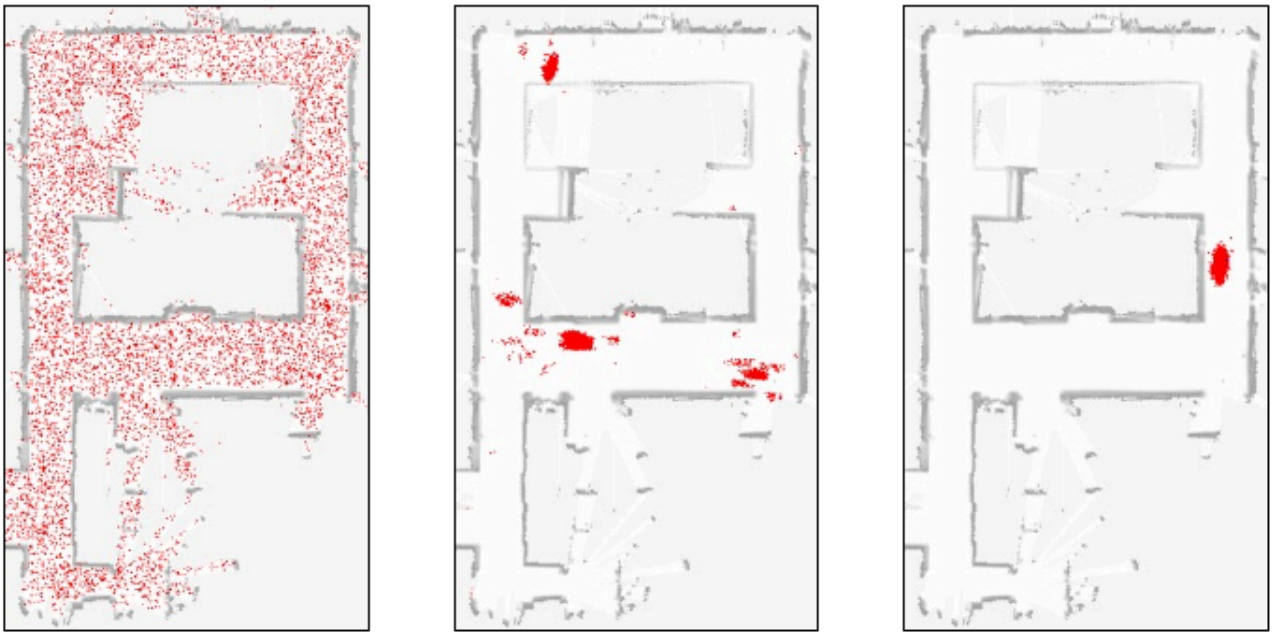
- **Selective Updating** to Increase Efficiency
  - Update only relevant parts of state space
    - No update when probability under certain threshold
  - Update all, only if robot is **de-localised**
    - likelihood in "relevant" parts of state space "small"
- **Delayed Motion Updates**
  - Apply motion update at **lower frequency**
  - **Accumulate odometry readings** over a short period

Summary

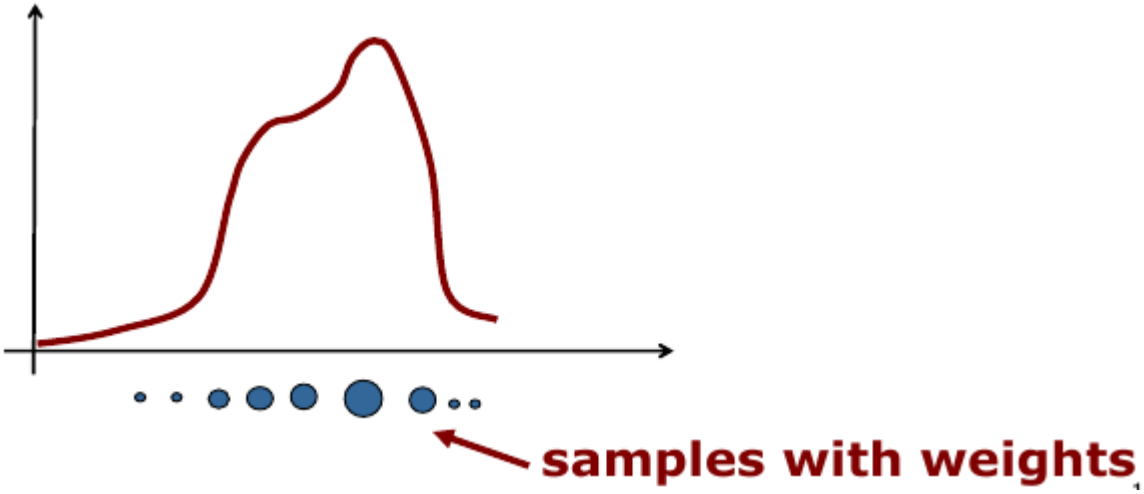
- Discretised belief about the state
- Histogram for representing density
- Naturally models **multi-modal** beliefs
- Performs **global localisation**, and recover from localisation errors
- Discretisation **limits pose accuracy**
- **All cell** require updates for motions or observations
- In practice: Approximations

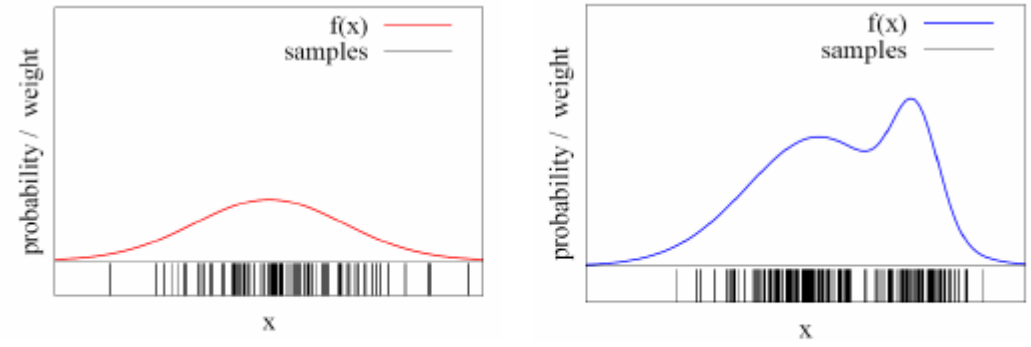
Particle Filter & Monte Carlo Localisation (MCL)

- Way to **efficiently** represent **non-Gaussian** distributions
  - Set of state hypotheses ("particles" or "samples")
  - Survival-of-the-fittest

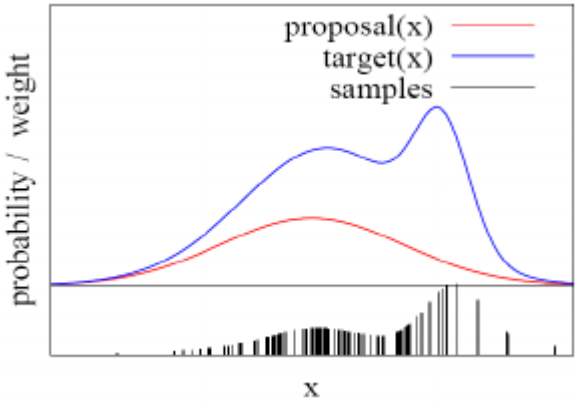
- 

Key Idea: Samples

- **Set of weighted samples** to represent arbitrary distributions
- 

- Particle Set
  - $\mathcal{X} = \{\langle \underbrace{x^{[j]}}_{\text{state hypothesis}}, \underbrace{w^{[j]}}_{\text{importance weight}} \rangle\}_{j=1,\ldots,J}$
  - For approximation
  - 

    The more samples fall into a region,
    the higher the probability of the region

- For Gaussian: $x \leftarrow \frac{1}{2}\sum_{i=1}^{12} rand(-\sigma, \sigma)$
- For other distributions:
  - Generate samples f (target)
  - from distribution π (proposal)
  - Pre-condition: $f(x) > 0 \rightarrow \pi(x) > 0$
  - **Weigth** for differences between π and f:
    - $w = \frac{f(x)}{\pi(x)}$
  - 

Particle Filter for Dynamic State Estimation

- Recursive Bayes filter
- Non-parametric
- Models the distribution by set of weighted samples
- **Prediction**: sample from the proposal
- **Correction**: weigh particles by the ratio of target and proposal
- **The more sample, the better the estimate**

Monte Carlo Localisation (MCL)

- **Each particle is a pose hypothesis**
- **Prediction**: For each particle, sample a new pose from the motion model (=proposal)

$$x_t^{[j]} \sim p(x_t | x_{t-1}^{[j]}, u_t)$$

- **Correction**: Weigh samples according to observation model (=target/proposal)

$$w_t^{[j]} \propto p(z_t | x_t^{[j]})$$

- **Resampling**: Draw sample i with probability $w_t^{[j]}$ and repeat j times (J=# particles)
- Odometry Motion Model:
  - Uncertainty in **translation & rotation** of the robot (Gaussian)
  - **For each particle**, draw a new pose by **sampling from three normal distribution**

- **Particle Weights** via Observation Model
  - Use beam-based model or end-point model
  - Compute **measurement likelyhood** by evaluating the actual observation w.r.t. the expected value
    - Assign it as **weight** of the particle

Particle Filter for Localisation

current set of particles

Particle_filter$(\mathcal{X}_{t-1}, u_t, z_t)$**:**

$\bar{\mathcal{X}}_t = \mathcal{X}_t = \emptyset$

for $j = 1$ *to* J *do* — for all particles

  sample $x_t^{[j]} \sim p(x_t \mid u_t, x_{t-1}^{[j]})$ — prediction

  $w_t^{[j]} = p(z_t \mid x_t^{[j]})$ — correction

  $\bar{\mathcal{X}}_t = \bar{\mathcal{X}}_t + \langle x_t^{[j]}, w_t^{[j]} \rangle$

endfor

for $j = 1$ *to* J *do* — resampling of particles

  draw $i \in 1, \ldots, J$ *with probability* $\propto w_t^{[i]}$

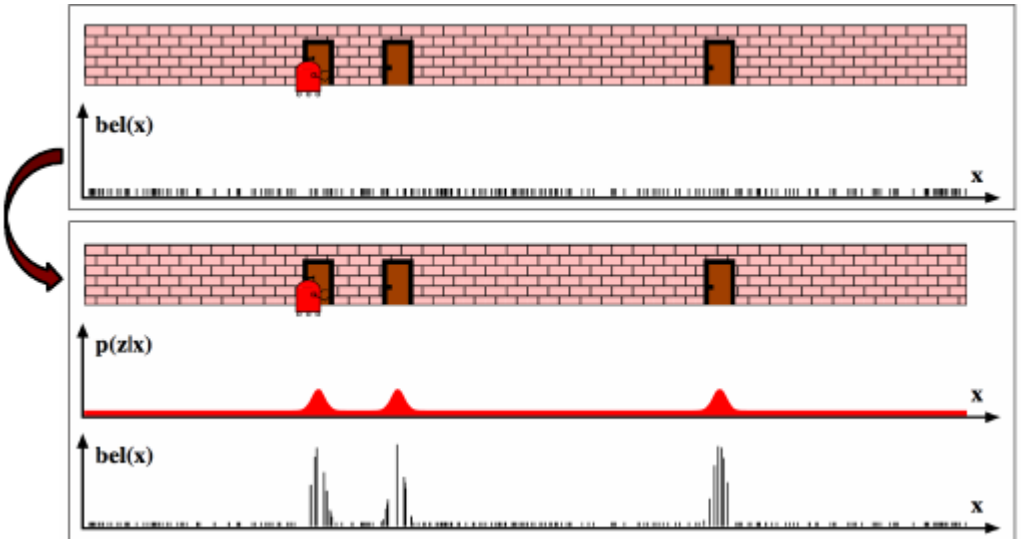
  add $x_t^{[i]}$ *to* $\mathcal{X}_t$

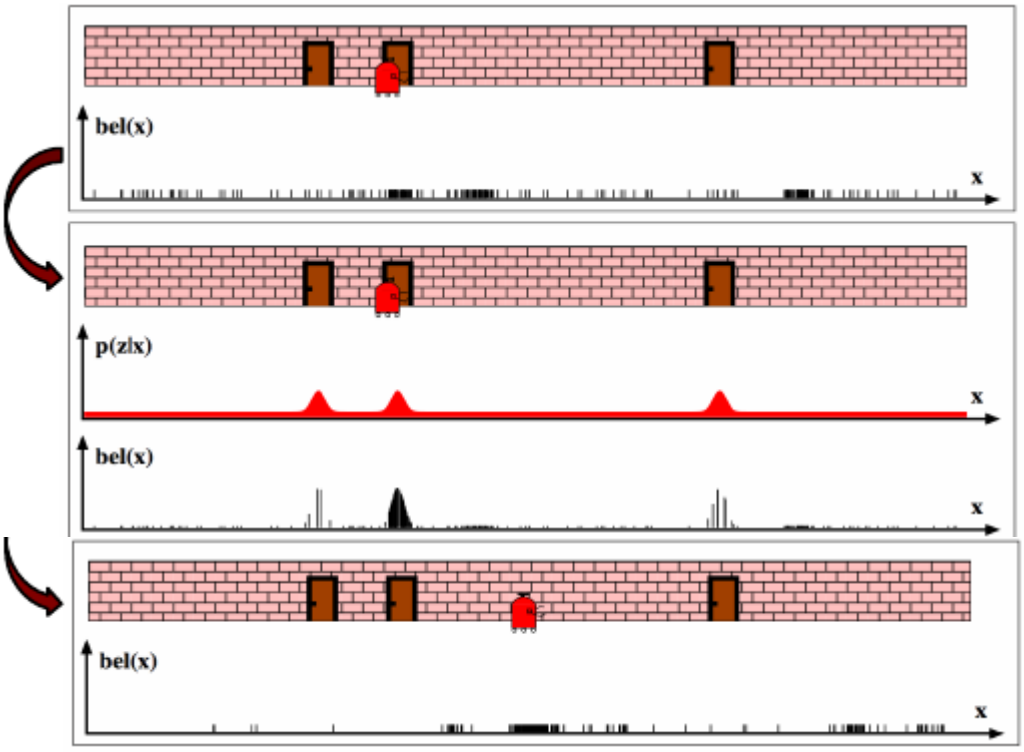
endfor

return $\mathcal{X}_t$ — new set of particles

MCL:

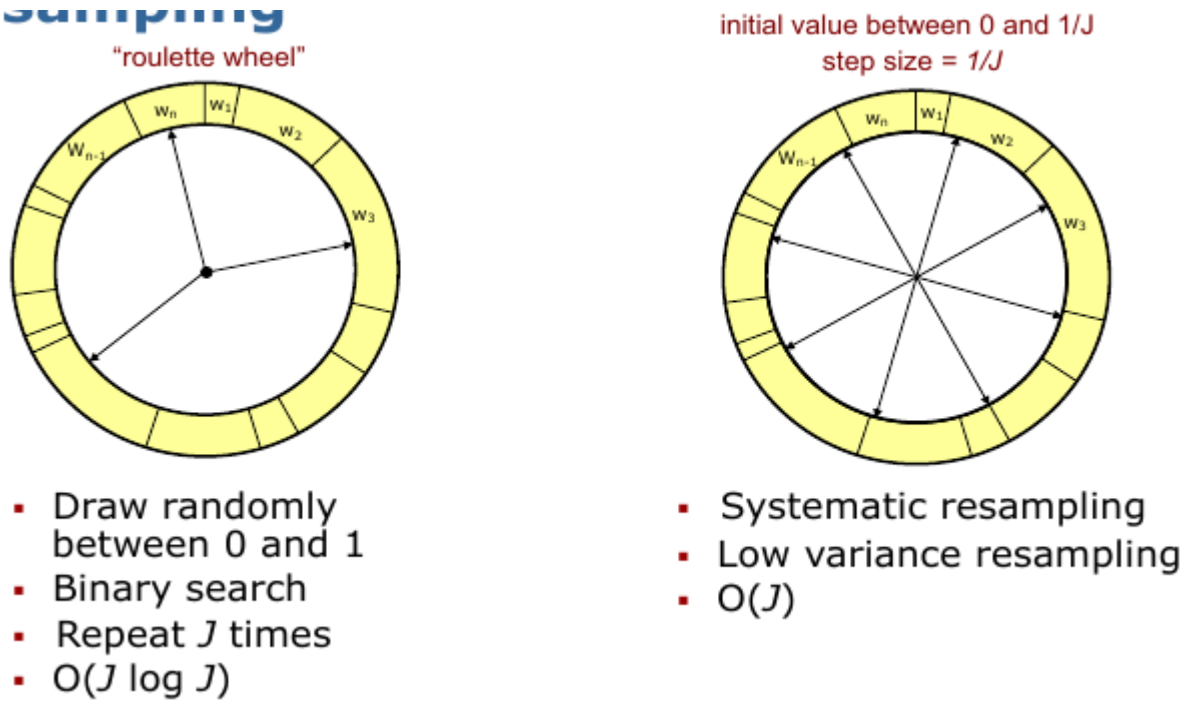
Correction Step-> Resampling and Prediction Step ->
Correction Ste->...





Resampling

- Repeat J times: Draw sample i with probability $w_t^{[i]}$ (J = # particles)
- Informally: "Replace unlikely samples by more likely ones"
  - Avoid too many samples in unlikely states
    - Only limited number of samples



- Roulette wheel resampling easy to understand but **suboptimal in practice**
  - All samples have the same weight?

Low Variance Resampling: Implementation

```
1. draw random number r between 0 and 1/J

2. for (j=1 .. J)
        U = r + (j-1) * 1/J
        while (U > cum[i])  i++;
        pick_particle i
```

- Keeps samples in case of same weights
- Faster than roulette wheel resampling (O(J) vs. O(J log J))
- **Always use low variance resampling**

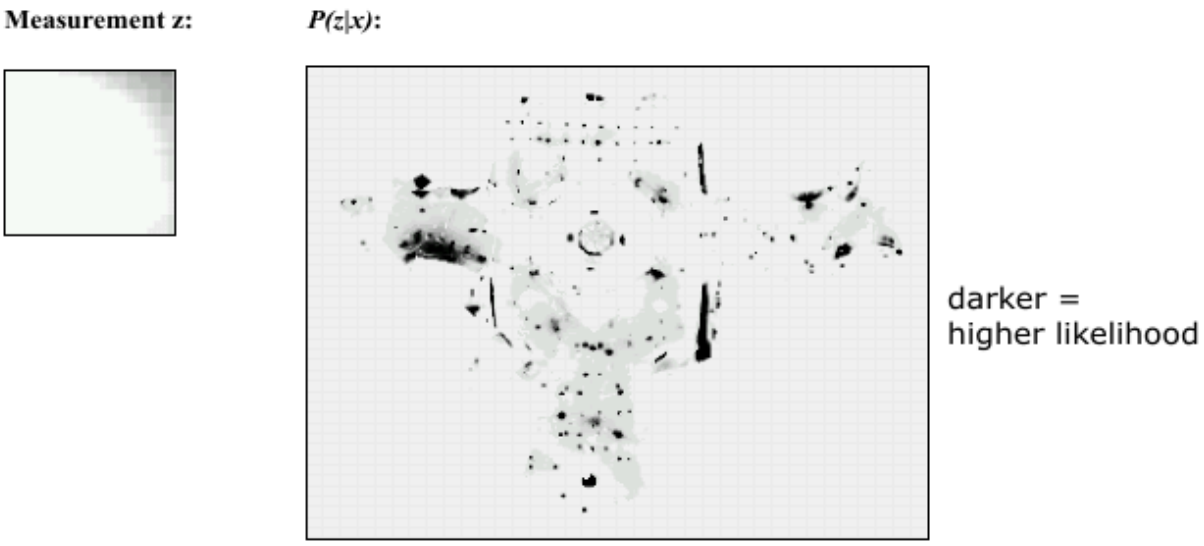
Localisation Error Recovery

- Error like Kidnapped robot problem
- At each time step randomly insert **fixed number of samples**
- OR, Insert number of random samples **proportional to average likelihood of particles**
- Without Random Sampling: No robot pose recovery

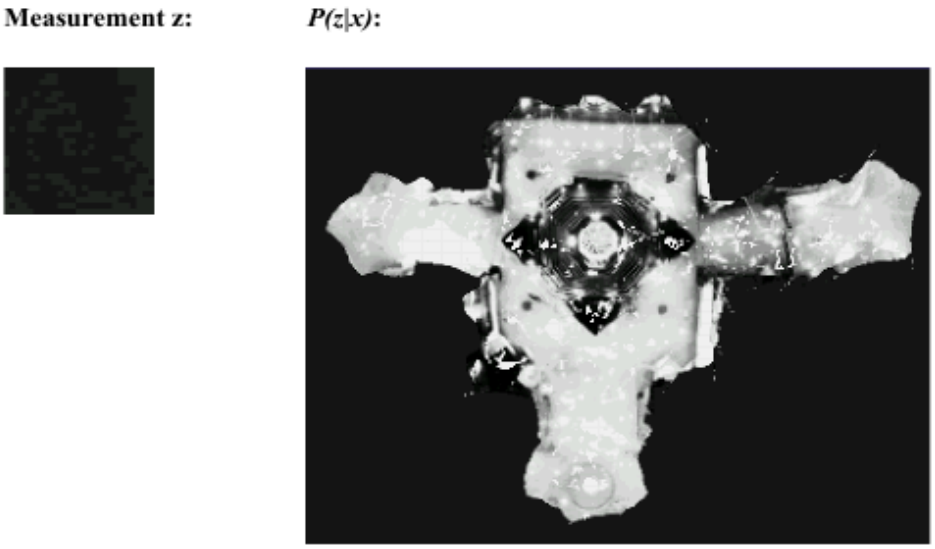
Ceiling Maps for Localisation

- Laser measurements affected by movement
- Use camera pointing upwards and ceiling map
  - Observation: intensity value of center area

Example: Under a Light



Example: No Light in the Center Area



Summary

General Particle Filter

- Particle filters are **non-parametric** Bayes filters
- Belief is represented by a set of **weighted samples**
- **Proposal distribution** to draw the samples for the next time step
- **Particle weight** to account for differences between proposal and the target
- Re-sampling: Drawing of new particles with a probability **proportional to the weights**

Monte Carlo Localisation

- Particles propagated according to the motion model
- Particles weighed according to **likelyhood fo observation**
- Easy to implement and robust

- Used in many practical localisation systems
- **Based on appropriate motion and sensors models**